

HavenWorld — Complete Build Guide

Part 3: Phase 1 Mac — Client, Phaser Scenes, Avatar Art & Room Maps

Developer	Daniel Boone
GitHub	github.com/dboone323/HavenWorld
Frontend URL	https://havenworld.pages.dev
Date	September 18, 2026
Series	Part 3 of 4 — Follows Part 2 (Windows: Repo, DB, Server scaffold)
Platform Scope	MacBook Pro — all steps tagged [MAC]
Document Category	Technical Build Guide — Game Industry / Indie Dev

Series Overview

Part	Title	Platform	Key Topics	Status
Part 1	Foundation & Architecture	Both	Monorepo, tech stack, Oracle VM, Cloudflare, infra overview	Complete

Part	Title	Platform	Key Topics	Status
Part 2	Phase 1 Windows — Repo, DB & Server Scaffold	Windows	Git, pnpm workspace, Prisma schema, Docker Postgres, server scaffold	Complete
Part 3	Phase 1 Mac — Client, Phaser, Avatar Art & Maps	Mac	Vite + Phaser 3, scenes, auth/socket services, avatar pipeline, Tiled maps	This Document
Part 4	Phase 1 Linux — Server, Auth, Socket Engine & Deploy	Linux (Oracle VM)	Prisma migrations, full auth API, Socket.io engine, PM2 deployment, alpha gate	Upcoming

Table of Contents

5	Phase 1 — Mac Tasks Overview	→ p.1
5.0	Phase 1 Goal & Estimated Timeline	
5.1	Initialize the Client App (Vite + Phaser 3) [MAC]	
	MAC-1-A1: Scaffold the Vite Project	
	MAC-1-A2: Install Client Dependencies	
	MAC-1-A3: Configure vite.config.ts	
	MAC-1-A4: Create Environment Variable Files	
	MAC-1-A5: Initialize apps/server Package	

5	Phase 1 — Mac Tasks Overview	→ p.1
5.2	Shared Package [MAC]	
	MAC-1-B1 through MAC-1-B4	
5.3	Phaser 3 Scene Architecture [MAC]	
	MAC-1-C1: Game Entry Point (main.ts)	
	MAC-1-C2: BootScene	
	MAC-1-C3: LoginScene	
	MAC-1-C4: LobbyScene	
	MAC-1-C5: RoomScene — Structure Overview	
	MAC-1-C6: UIScene	
5.4	Auth and Socket Service Modules [MAC]	
	MAC-1-D1: Auth Service (src/services/auth.ts)	
	MAC-1-D2: Socket Service (src/services/socket.ts)	
5.5	HTML Game Shell (index.html) [MAC]	
	MAC-1-E1: Replace Default index.html	
	MAC-1-E2: Create the CSS Stylesheet (style.css)	
5.6	Avatar Art Pipeline [MAC]	
	MAC-1-F1: Install LibSprite & TexturePacker	
	MAC-1-F2: Avatar Frame Specification	
	MAC-1-F3: Draw the Minimum Viable Avatar Sprite Set	
	MAC-1-F4: Pack Layers into a Phaser Atlas (TexturePacker)	
	MAC-1-F5: Register Avatar	

5	Phase 1 — Mac Tasks Overview	→ p.1
	Animations in Phaser	
	MAC-1-F6: Create the Avatar Entity Class	
5.7	Room Map Creation with Tiled [MAC]	
	MAC-1-G1: Install & Set Up Tiled Map Editor	
	MAC-1-G2 through G6: Create All 5 Maps	
	MAC-1-G7: Load Tilemaps in Phaser (RoomScene)	
5.8	Click-to-Move with EasyStar.js Pathfinding [MAC]	
	MAC-1-H1: Build the Walkability Grid	
	MAC-1-H2: Handle Click-to-Move Input	
	MAC-1-H3: Walk the Avatar Along the Path (update())	
5.9	Avatar Customizer UI [MAC]	
	MAC-1-I1: AvatarCustomizer Module	
	MAC-1-I2: Wire Up Avatar Panel Buttons	
5.10	Phase 1 Mac Completion Checklist [MAC]	

Section 5: Phase 1 — Mac Tasks [MAC]

Phase 1 Goal: Build the game so **5-25 alpha testers** can log in, create an avatar, enter a room, move around in real time, and chat with other players. This document covers all Mac-side work. Estimated time for Phase 1 Mac: **3-5 weeks** of focused development. All steps in this document are tagged **[MAC]** and are intended to be performed on a MacBook Pro. Linux/server tasks are covered in Part 4.

5.1 Initialize the Client App (Vite + Phaser 3) [MAC]

The HavenWorld frontend is a Vite-bundled TypeScript application using Phaser 3 as its game engine. Vite provides lightning-fast hot module replacement during development and optimized chunking for production. Phaser 3 handles all canvas rendering, animation, input, physics, and scene management. This section scaffolds the client from scratch inside the monorepo.

MAC-1-A1 | Scaffold the Vite Project [MAC]

From the HavenWorld repo root in Terminal:

```
cd apps/client pnpm create vite@latest . -- --template vanilla-ts
```

When prompted, confirm overwriting the empty directory. Then install dependencies:

```
pnpm install
```

Verify the scaffold succeeded:

```
ls src/ # Expected output: main.ts style.css vite-env.d.ts
```

MAC-1-A2 | Install Client Dependencies [MAC]

```
pnpm add phaser socket.io-client alpinejs pnpm add -D @types/node vite-plugin-static-copy easystarjs
```

NOTE — Bundle Size

Phaser 3 is approximately **1 MB minified**. We chunk it separately in the Rollup configuration (see MAC-1-A3) so the initial page load is fast and Phaser can be cached by the browser independently of application code.

MAC-1-A3 | Configure vite.config.ts [MAC]

Replace the default `apps/client/vite.config.ts` with the full configuration below. The proxy forwards all `/api` and `/socket.io` requests to the local Node.js server during development, so you never need CORS exceptions in dev. In production, `VITE_SERVER_URL` points directly to the Oracle VM's nip.io URL.

```
import { defineConfig } from 'vite'; import { viteStaticCopy } from 'vite-plugin-static-copy'; export default defineConfig({  server: {    port: 5173,    proxy: {      '/api': { target: 'http://localhost:3000', changeOrigin: true },      '/socket.io': { target: 'http://localhost:3000', ws: true, changeOrigin: true },    },  },  build: {    rollupOptions: {      output: {        manualChunks: {          phaser: ['phaser'],        },      },    },    plugins: [      viteStaticCopy({ targets: [{ src: 'src/assets', dest: '.' }] })    ],  });
```

MAC-1-A4 | Create Environment Variable Files [MAC]

Create `apps/client/.env.development`

```
VITE_SERVER_URL= # Empty – the Vite proxy handles routing in development. # An empty string causes the client to use the current origin.
```

Create `apps/client/.env.production`

```
VITE_SERVER_URL=https://YOUR-ORACLE-IP-WITH-DASHES.nip.io # Replace with your actual nip.io URL after the Oracle VM is provisioned (see Part 4).
```

SECURITY — Add to .gitignore

Add both `.env.development` and `.env.production` to your root `.gitignore`. Never

commit production URLs or secrets. Also add the `TYPE_SERVER_URL` to Cloudflare Pages: **Pages** → **havenworld** → **Settings** → **Environment Variables** → **Production**. Set the value to your nip.io URL so Cloudflare builds pick it up automatically.

MAC-1-A5 | Initialize apps/server Package (Mac-Side Scaffolding) [MAC]

Even though the server will ultimately run on the Oracle Linux VM, you scaffold it on the Mac so that TypeScript path resolution and the shared package link work across the entire monorepo. Run from the repo root:

```
cd ../server pnpm init pnpm add express socket.io @prisma/client bcryptjs jsonwebtoken pnpm add cookie-parser cors helmet morgan express-rate-limit bad-words pnpm add @sentry/node resend zod redis pnpm add -D typescript @types/node @types/express @types/bcryptjs pnpm add -D @types/jsonwebtoken @types/cookie-parser jest ts-jest nodemon ts-node pnpm add -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Create `apps/server/tsconfig.json`

```
{  "compilerOptions": {    "target": "ES2022",    "module": "CommonJS",    "lib": ["ES2022"],    "outDir": "./dist",    "rootDir": "./src",    "strict": true,    "esModuleInterop": true,    "resolveJsonModule": true,    "skipLibCheck": true,    "paths": {      "@shared/*": ["../../packages/shared/src/*"]    },    "include": ["src/**/*.ts", "prisma/seed.ts"],    "exclude": ["node_modules", "dist"]  }
```

5.2 Shared Package [MAC]

The `@havenworld/shared` package contains TypeScript interfaces and socket event name constants that are imported by *both* the client and the server. Keeping these in a shared package guarantees that both sides of the WebSocket connection always agree on event names and data shapes — a single source of truth. Because it is a workspace package, changes made here are instantly reflected in both `apps/client` and `apps/server` without any publish step.

MAC-1-B1 | Initialize packages/shared [MAC]

```
cd ../../packages/shared pnpm init pnpm add -D typescript
```

Create `packages/shared/package.json` (replace the generated one):

```
{  "name": "@havenworld/shared",  "version": "0.1.0",  "main": "src/index.ts",  "exports": {    ".": "./src/index.ts"  } }
```

MAC-1-B2 | Create Socket Event Constants (src/events.ts) [MAC]

Using constants instead of raw strings prevents hard-to-find typos where the client emits `player:moved` but the server listens for `player:move`. TypeScript's `as const` ensures the values are narrowed to string literals, enabling full IntelliSense autocomplete when calling `socketService.emit()` or `socketService.on()`.

```
export const SOCKET_EVENTS = {  AUTH_JOIN:      'auth:join',  AUTH_ERROR:      'auth:error',  ROOM_STATE:      'room:state',  ROOM_PLAYER_JOINED: 'room:player_joined',  ROOM_PLAYER_LEFT: 'room:player_left',  PLAYER_MOVE:      'player:move',  PLAYER_POSITION:  'player:position',  CHAT_SEND:        'chat:send',  CHAT_MESSAGE:     'chat:message',  CHAT_ERROR:       'chat:error',  AVATAR_UPDATE:    'avatar:update',  AVATAR_CHANGED:   'avatar:changed',  FRIEND_ONLINE:    'friend:online',  FRIEND_OFFLINE:   'friend:offline', } as const;
```

MAC-1-B3 | Create Shared TypeScript Interfaces (src/types.ts) [MAC]

These interfaces describe all the core data shapes that travel over the network. Defining them once here ensures that the server's socket handlers and the client's scene code use identical types, and TypeScript will catch any mismatches at compile time rather than at runtime.

```
export interface AvatarData {  bodyType: string;  skinTone: string;  hairstyle: string;  hairColor: string;  eyeStyle: string;  outfitHead: string | null;  outfitFace: string | null;  outfitBody: string | null;  outfitLegs: string | null;  outfitFeet: string | null;  outfitBack: string | null;  outfitHand: string | null; }  export interface PlayerState {  id: string;  username: string;  avatar: AvatarData;  x: number;  y: number;  direction: 'down' | 'up' | 'left' | 'right';  isMoving: boolean; }  export interface ChatMessage {  id: string;  senderId: string;  senderName: string;  content: string;  isFiltered: boolean;  timestamp: string; }  export interface FurnitureState {  id: string;  itemId: string; }
```



```

spriteKey: string;  x:      number;  y:      number;  z:
number;  rotation:  number;  layer:    number; } export interface
RoomData {  id:      string;  name:      string;  width:
number;  height:    number;  backgroundKey: string;  players:
PlayerState[];  furniture:  FurnitureState[];  chatHistory:
ChatMessage[]; }

```

MAC-1-B4 | Create src/index.ts and Link to Other Packages [MAC]

Create `packages/shared/src/index.ts`

```
export * from './events'; export * from './types';
```

In both `apps/client/package.json` and `apps/server/package.json` add to the `dependencies` section:

```
"@havenworld/shared": "workspace:*"
```

From the repo root, run `npm install` to create the symlink. After this, any file in client or server can import from the shared package using:

```
import { SOCKET_EVENTS, PlayerState } from '@havenworld/shared';
```

5.3 Phaser 3 Scene Architecture [MAC]

HavenWorld uses five Phaser scenes. Each scene has a single responsibility. Scenes are registered once in the Phaser `GameConfig` and then started or stopped programmatically as the player progresses through the app. The `BootScene` always runs first to preload all shared assets. The `UIScene` is unique in that it runs in *parallel* with `RoomScene` — Phaser supports multiple simultaneously active scenes, and this pattern is the standard way to render a persistent HUD on top of the game world without interference.

Scene Key	Responsibility	Runs With	Triggers Next
<code>BootScene</code>	Preload ALL game assets — tilemaps, atlases, tilesets	Alone	LoginScene
<code>LoginScene</code>	Show HTML login/register overlay; hide Phaser canvas	Alone	LobbyScene (after auth)

Scene Key	Responsibility	Runs With	Triggers Next
LobbyScene	Fetch room list; render clickable room cards in HTML	Alone	RoomScene + UIScene
RoomScene	Render tilemap, all avatars, handle socket events and pathfinding	UIScene (parallel)	—
UIScene	Chat input/output, HUD button wiring — overlays RoomScene	RoomScene (parallel)	—

MAC-1-C1 | Phaser Game Entry Point (apps/client/src/main.ts) [MAC]

Replace the default `apps/client/src/main.ts` `Phaser.AUTO` automatically selects WebGL if the browser supports it, and falls back to Canvas otherwise. The background color `#1a1a2e` matches the dark navy theme of the HTML shell so there is no color flash between the HTML overlay hiding and the canvas appearing.

```
import Phaser from 'phaser'; import { BootScene } from './scenes/BootScene';
import { LoginScene } from './scenes/LoginScene'; import { LobbyScene } from
 './scenes/LobbyScene'; import { RoomScene } from './scenes/RoomScene';
import { UIScene } from './scenes/UIScene'; const config:
Phaser.Types.Core.GameConfig = { type: Phaser.AUTO, width:
1024, height: 768, backgroundColor: '#1a1a2e', parent:
'game-container', physics: { default: 'arcade', arcade: { gravity:
{ y: 0 }, debug: false }, }, scene: [BootScene, LoginScene, LobbyScene,
RoomScene, UIScene], }; new Phaser.Game(config);
```

MAC-1-C2 | BootScene (scenes/BootScene.ts) [MAC]

The BootScene uses Phaser's built-in asset loader to fetch all game assets before any gameplay begins. A loading progress bar gives players visual feedback. All tilemaps and sprite atlases loaded here are available to every subsequent scene without re-fetching. Create `src/scenes/BootScene.ts`

```
import Phaser from 'phaser'; export class BootScene extends Phaser.Scene {
constructor() { super({ key: 'BootScene' }); } preload() { // Loading
```

```

progress bar      const bar = this.add.graphics();
this.load.on('progress', (v: number) =>
{
    bar.clear().fillStyle(0x4ecdc4).fillRect(200, 370, 624 * v,
20);
}); // Tilemaps (JSON exported from Tiled)
this.load.tilemapTiledJSON('map-lobby', 'assets/maps/lobby.json');
this.load.tilemapTiledJSON('map-town-square', 'assets/maps/town-
square.json'); this.load.tilemapTiledJSON('map-park',
'assets/maps/park.json'); this.load.tilemapTiledJSON('map-cafe',
'assets/maps/cafe.json'); this.load.tilemapTiledJSON('map-personal-room',
'assets/maps/personal-room.json'); // Tilesets (PNG images referenced by
tilemaps) this.load.image('tileset-outdoor',
'assets/sprites/tilesets/outdoor.png'); this.load.image('tileset-indoor',
'assets/sprites/tilesets/indoor.png'); // Avatar sprite atlas (all
layers packed into one PNG + atlas JSON)
this.load.atlas('avatars', 'assets/sprites/avatar/atlas.png',
'assets/sprites/avatar/atlas.json'); // Furniture sprite atlas
this.load.atlas('furniture',
'assets/sprites/furniture/atlas.png',
'assets/sprites/furniture/atlas.json'); } create()
{ this.scene.start('LoginScene'); } }

```

MAC-1-C3 | LoginScene (scenes/LoginScene.ts) [MAC]

The LoginScene itself is minimal — its only job is to reveal the HTML login panel. All form validation, fetch calls, token storage, and error display are wired up in `src/ui/loginUI.ts` which is initialized in `main.ts` before the Phaser game starts. This keeps auth logic completely outside the Phaser lifecycle. Create

`src/scenes/LoginScene.ts`:

```

import Phaser from 'phaser'; export class LoginScene extends Phaser.Scene {
constructor() { super({ key: 'LoginScene' }); } create() { // Reveal
the HTML login panel defined in index.html
document.getElementById('login-panel')!.style.display = 'flex'; } }

```

MAC-1-C4 | LobbyScene (scenes/LobbyScene.ts) [MAC]

After a successful login, the LobbyScene fetches the list of public rooms and renders them as clickable HTML cards. When a player clicks a room card, the lobby panel hides, RoomScene starts, and UIScene launches in parallel (the second scene runs on top of the first without stopping it). Create `src/scenes/LobbyScene.ts`

```

import Phaser from 'phaser'; import { authService } from '../services/auth';
export class LobbyScene extends Phaser.Scene { constructor() { super({ key:
'LobbyScene' }); } async create() { const panel =
document.getElementById('lobby-panel')!; panel.style.display = 'block';

```

```
const rooms = await fetch('/api/rooms', { headers: { Authorization:
`Bearer ${AuthService.getToken()}` }, }).then(r => r.json());
panel.innerHTML = '<h2>Choose a Room</h2>' + rooms.map((room: any) => `
<div class="room-card" data-room-id="${room.id}">
<h3>${room.name}</h3>      <span class="occupants">${room.occupants} / $
{room.maxOccupants} players</span>      </div>      `).join('');
panel.querySelectorAll('.room-card').forEach(card =>
{      card.addEventListener('click', () => {      const roomId =
card.getAttribute('data-room-id')!;      panel.style.display = 'none';
this.scene.start('RoomScene', { roomId });
this.scene.launch('UIScene', { roomId }); // UIScene runs in parallel
});      });      }
```

MAC-1-C5 | RoomScene (scenes/RoomScene.ts) — Structure Overview [MAC]

The RoomScene is the largest and most complex scene in the project. It owns the tilemap rendering, all avatar entities, EasyStar pathfinding, and all real-time socket event handling. The following is the authoritative structural specification. Full implementation details for pathfinding are in Section 5.8; avatar entities are in Section 5.6.

Method	Responsibilities
<code>init(data)</code>	Store <code>roomId</code> from scene launch data for use in <code>create()</code>
<code>preload()</code>	Nothing — all assets loaded in BootScene
<code>create()</code>	1. Load tilemap using <code>this.make.tilemap({ key: map-\${roomId} })</code> 2. Add tilesets and create Ground, Walls, Objects, Overlay layers 3. Set collision: <code>wallsLayer.setCollisionByProperty({ collides: true })</code> 4. Build EasyStar walkability grid from Walls layer (see MAC-1-H1) 5. Call <code>socketService.connect()</code> and emit <code>auth:join</code> with <code>roomId</code> 6. Listen for <code>room:state</code> → create Avatar entity for each player 7. Listen for <code>room:player_joined</code> → create new Avatar entity 8. Listen for <code>room:player_left</code> → destroy that Avatar entity 9. Listen for <code>player:position</code> →

Method	Responsibilities
	call <code>avatar.moveTo(x, y, direction)</code> 10. Listen for <code>avatar:changed</code> → call <code>avatar.updateAvatar/avatarData</code> 11. Listen for <code>chat:message</code> → call <code>avatar.showSpeechBubble(content)</code> 12. Set up click-to-move pointer input (see Section 5.8) 13. Set camera: <code>this.cameras.main.startFollow(localAvatar, true, 0.1, 0.1)</code> 14. Set camera bounds to tilemap pixel dimensions 15. Update room name HUD: <code>document.getElementById('room-name').textContent = roomName</code>
<code>update()</code>	1. Depth-sort all avatars by Y position: <code>avatars.forEach(a => a.setDepth(a.y))</code> 2. Call <code>easyStar.calculate()</code> each frame to process queued path requests 3. Advance localAvatar along current walkPath, emit <code>player:move</code> per tile step

Key properties on the RoomScene class: `map: Phaser.Tilemaps.Tilemap`, `avatars: Map<string, Avatar>` (keyed by socket ID), `localAvatar: Avatar`, `easyStar: EasyStar.js`, `currentRoomId: string`, `walkPath: {x,y}[]`, `pathindex: number`.

MAC-1-C6 | UIScene (scenes/UIScene.ts) [MAC]

UIScene runs simultaneously with RoomScene and manages the chat panel and HUD buttons. Because it is a separate scene, it has its own update loop and can be paused/resumed independently. Create `src/scenes/UIScene.ts`

```
import Phaser from 'phaser'; import { socketService } from
'../services/socket'; import { SOCKET_EVENTS } from '@havenworld/shared';
export class UIScene extends Phaser.Scene { constructor() { super({ key:
'UIScene' }); } create() { const chatPanel =
document.getElementById('chat-panel')!; chatPanel.style.display = 'flex';
const input = document.getElementById('chat-input') as HTMLInputElement;
const sendBtn = document.getElementById('chat-send')!; const sendMessage
= () => { const content = input.value.trim(); if (!content)
return; socketService.emit(SOCKET_EVENTS.CHAT_SEND, { content });
input.value = ''; }; sendBtn.addEventListener('click', sendMessage);
input.addEventListener('keydown', e => { if (e.key === 'Enter')
sendMessage(); }); socketService.on(SOCKET_EVENTS.CHAT_MESSAGE, (msg:
any) => { const history = document.getElementById('chat-history')!;
const el = document.createElement('div'); el.className = 'chat-
```

```
message';      el.innerHTML =      `<span class="chat-sender">$
{msg.senderName}</span> ${msg.content}`;      history.appendChild(el);
history.scrollTop = history.scrollHeight;      });
socketService.on(SOCKET_EVENTS.CHAT_ERROR, (err: any) => {      if (err.code
=== 'RATE_LIMITED') {      document.getElementById('chat-
error')!.textContent = 'Slow down!';      setTimeout(() =>
{      document.getElementById('chat-error')!.textContent =
'';      }, 2000);      }      });  } }
```

5.4 Auth and Socket Service Modules [MAC]

The auth and socket modules are plain TypeScript singleton objects — not Phaser classes. They live in `src/services/` and are imported by scenes, UI modules, and the HTML shell alike. Keeping them outside of Phaser's scene lifecycle means they survive scene transitions and retain their state (access token, socket connection) across the entire session.

MAC-1-D1 | Auth Service (`src/services/auth.ts`) [MAC]

The access token is stored in a **module-scoped variable** (not `localStorage` or a cookie). This prevents XSS attacks from reading the token via injected JavaScript. The refresh token is an `HttpOnly` cookie managed by the browser — JavaScript cannot access it. The service silently refreshes the access token every 13 minutes; tokens expire at 15 minutes, giving a 2-minute buffer. Create `src/services/auth.ts`

```
const SERVER = import.meta.env.VITE_SERVER_URL ?? ''; let accessToken: string
| null = null; let currentUser: { id: string; username: string; role:
string } | null = null; export const authService = {  async
register(      username: string, email: string,      password: string,
inviteCode: string  ) {      const res = await
fetch(`${SERVER}/api/auth/register`, {      method: 'POST', credentials:
'include',      headers: { 'Content-Type': 'application/json' },      body:
JSON.stringify({ username, email, password, inviteCode }),      });      return
res.json();  },  async login(email: string, password: string) {      const
res = await fetch(`${SERVER}/api/auth/login`, {      method: 'POST',
credentials: 'include',      headers: { 'Content-Type':
'application/json' },      body: JSON.stringify({ email,
password }),      });      const data = await res.json();      if
(data.accessToken) {      accessToken = data.accessToken;      currentUser
= data.user;      }      return data;  },  async refresh() {      const res =
await fetch(`${SERVER}/api/auth/refresh`, {      method: 'POST',
credentials: 'include',      });      const data = await res.json();      if
(data.accessToken) accessToken = data.accessToken;      return !!
data.accessToken;  },  async logout() {      await
fetch(`${SERVER}/api/auth/logout`, {      method: 'POST', credentials:
```

```
'include',    });    accessToken = null;    currentUser = null;    },
getToken:    () => accessToken,    getUser:    () => currentUser,    isLoggedIn:
() => !!accessToken, }); // Silently refresh the access token every 13 min
(token expires at 15 min) setInterval(async () => {    if (accessToken) await
authService.refresh(); }, 13 * 60 * 1000);
```

MAC-1-D2 | Socket Service (src/services/socket.ts) [MAC]

The SocketService is a singleton class. All Phaser scenes and UI modules import the same `socketService` instance, so there is only ever one WebSocket connection per client session. The `connect()` method guards against creating duplicate connections (checking `socket?.connected` before creating a new one). Reconnection is handled automatically by Socket.io with up to 5 attempts and 1-second initial delay. Create

`src/services/socket.ts`:

```
import { io, Socket } from 'socket.io-client'; import { authService } from
'./auth'; const SERVER = import.meta.env.VITE_SERVER_URL ?? ''; class
SocketService {    private socket: Socket | null = null;    connect() {    if
(this.socket?.connected) return; // Guard: never open duplicate connections
this.socket = io(SERVER, {    auth:    { token:
authService.getToken() },    transports:    ['websocket',
'polling'],    reconnectionAttempts: 5,    reconnectionDelay:    1000,
});    this.socket.on('connect', () =>    console.log('[Socket]
connected:', this.socket!.id));    this.socket.on('connect_error', err =>
console.error('[Socket] error:', err.message));
this.socket.on('disconnect', reason =>    console.warn('[Socket]
disconnected:', reason));    }    emit(event: string, data?: any)
{ this.socket?.emit(event, data); }    on(event: string, handler: (data: any)
=> void)    { this.socket?.on(event, handler); }    off(event: string,
handler?: (data: any) => void)    { this.socket?.off(event, handler); }
disconnect() { this.socket?.disconnect(); this.socket = null; }
isConnected() { return this.socket?.connected ?? false; } } export const
socketService = new SocketService();
```

5.5 HTML Game Shell (index.html) [MAC]

The HTML shell is the single-page application container. All game UI panels — login, lobby, chat, avatar customizer, HUD — live here as hidden `div` elements. Phaser renders its canvas inside `#game-container`, and individual panels are shown or hidden by setting their CSS `display` property from JavaScript as game state changes. This hybrid HTML-over-canvas approach gives you full control over UI styling without being limited by Phaser's DOM Text objects.

MAC-1-E1 | Replace the Default index.html [MAC]

Replace `apps/client/index.html` entirely. The following table documents all panels and their IDs — every ID here is referenced in code:

Element ID	Default Display	Purpose	Shown By
<code>#game-container</code>	block	Phaser canvas mount point	Phaser (auto)
<code>#login-panel</code>	none	Login + register tabs and forms	LoginScene.create()
<code>#lobby-panel</code>	none	Room selection grid	LobbyScene.create()
<code>#chat-panel</code>	none	In-game chat overlay (bottom-left)	UIScene.create()
<code>#avatar-panel</code>	none	Avatar customizer UI	avatarCustomizer.open()
<code>#room-nav</code>	none	Top HUD: room name + nav buttons	RoomScene.create()
<code>#player-card</code>	none	Popup when clicking another player	Avatar click handler

login-panel must include: a login tab form (email, password, submit "Enter HavenWorld", `#login-error`) and a register tab form (username, email, password, invite code, submit "Create Account", `#register-error`). Tab switching is handled in `src/ui/loginUI.ts`

chat-panel must include: `#chat-history` (scrollable), `#chat-input` (text, maxlength 200), `#chat-send` (button), and `#chat-error` (rate-limit message).

avatar-panel must include: `#avatar-preview`, `#avatar-tabs`, `#avatar-items-grid`, `#avatar-save`, `#avatar-close`.

room-nav must include: `#room-name-display`, `#btn-world-map` ( World), `#btn-my-room` ( My Room), `#btn-avatar` ( Avatar).

player-card must include: `#player-card-avatar-preview`, `#player-card-username`, `#player-card-visit` (Visit Room), `#player-card-close` (×).

The closing `<script>` tag must be: `<script type="module" src="/src/main.ts"></script>`

MAC-1-E2 | Create the CSS Stylesheet (src/style.css) [MAC]

The stylesheet defines the dark navy/teal visual theme for all HTML panels. Key style rules are documented below. The actual file will be substantially longer — every panel and its inner elements must be covered.

Selector	Key Properties
<code>body</code>	<code>margin: 0; overflow: hidden; background: #0d0d1a; font-family: system-ui, sans-serif; color: white</code>
<code>#game-container</code>	<code>position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; z-index: 1</code>
<code>#login-panel</code>	<code>position: fixed; z-index: 100; width/height: 100%; display: flex; align-items: center; justify-content: center; background: rgba(0,0,0,0.85)</code>
<code>.panel-box</code>	<code>width: 400px; background: rgba(10,10,30,0.95); border: 1px solid rgba(78,205,196,0.4); border-radius: 16px; padding: 40px</code>
<code>h1</code> (in panels)	<code>color: #4ecdc4; text-align: center; margin-bottom: 24px; font-size: 2rem</code>
<code>.tab</code>	<code>background: none; border: none; color: #aaa; cursor: pointer; padding: 8px 16px; font-size: 1rem</code>
<code>.tab.active</code>	<code>color: #4ecdc4; border-bottom: 2px solid #4ecdc4</code>
<code>input[type=text/email/password]</code>	<code>width: 100%; padding: 10px 12px; background: #1a1a3e; border: 1px solid #333; color: white; margin-bottom: 12px; box-sizing: border-box</code>
<code>input:focus</code>	<code>outline: none; border-color: #4ecdc4</code>

Selector	Key Properties
<code>button[type=submit]</code>	width: 100%; padding: 12px; background: #4ecdc4; color: #0d0d1a; border: none; font-weight: bold; cursor: pointer
<code>.error</code>	color: #ff6b6b; font-size: 0.8rem; margin-top: 8px
<code>.room-card</code>	background: rgba(78,205,196,0.1); border: 1px solid rgba(78,205,196,0.3); padding: 20px; margin: 12px; cursor: pointer; display: inline-block; width: 220px; vertical-align: top; transition: transform 0.2s
<code>.room-card:hover</code>	transform: translateY(-4px); border-color: #4ecdc4
<code>#chat-panel</code>	position: fixed; bottom: 16px; left: 16px; width: 320px; z-index: 50; background: rgba(0,0,0,0.75)
<code>#chat-history</code>	height: 180px; overflow-y: auto; padding: 8px; display: flex; flex-direction: column; gap: 4px
<code>.chat-sender</code>	color: #4ecdc4; font-weight: bold
<code>#room-nav</code>	position: fixed; top: 16px; right: 16px; z-index: 50; display: flex; gap: 8px; align-items: center
<code>.nav-btn</code>	background: rgba(78,205,196,0.2); border: 1px solid rgba(78,205,196,0.4); color: white; border-radius: 20px; padding: 6px 14px; cursor: pointer; font-size: 0.85rem

5.6 Avatar Art Pipeline [MAC]

The avatar system uses a **layered sprite composition** model: each clothing item, hair style, eye style, and body type is a separate transparent PNG spritesheet. In-game, multiple Phaser sprites are stacked at the same position inside a `Container` object — one sprite per visible layer. This allows any combination of items to be mixed and matched at runtime without pre-rendering every combination.

TexturePacker combines all layer spritesheets into a single GPU texture atlas, minimizing draw calls.

MAC-1-F1 | Install LibreSprite and TexturePacker [MAC]

LibreSprite is a free open-source pixel art editor — a fork of the original Aseprite with the same workflow and shortcuts. Download the macOS `.dmg` from github.com/LibreSprite/LibreSprite/releases, open it, drag LibreSprite to Applications, and open from Applications. Use LibreSprite for all sprite drawing tasks in this section.

TexturePacker (free tier) packs multiple PNG spritesheets into a single texture atlas compatible with Phaser 3's JSON Hash format. The free tier supports up to 2 sprite sheets — sufficient for Phase 1 alpha (1 avatar atlas + 1 furniture atlas). Download from codeandweb.com/texturepacker, install, and open.

MAC-1-F2 | Avatar Frame Specification [MAC]

Establish these dimensions *before drawing anything*. All layers must match exactly — any deviation means frames will misalign when stacked on top of each other during rendering.

Parameter	Value	Notes
Frame width	64 px	Pixels per frame, horizontal
Frame height	96 px	Pixels per frame, vertical
Directions	4	down, up, left, right
Walk frames per direction	4	16 walk frames total per layer
Idle frames per direction	1	4 idle frames total per layer
Total frames per layer	20	4 idle + 16 walk

Parameter	Value	Notes
Spritesheet layout	Horizontal strip	All 20 frames in one row
Spritesheet size per layer	1280 × 96 px	64 × 20 = 1280 wide
Background	Transparent	RGBA PNG — no white background

Frame order within each horizontal strip (left to right):

Index 0: idle-down	Index 4: walk-down-0	Index 8: walk-up-0
Index 1: idle-up	Index 5: walk-down-1	Index 9: walk-up-1
Index 2: idle-left	Index 6: walk-down-2	Index 10: walk-up-2
Index 3: idle-right	Index 7: walk-down-3	Index 11: walk-up-3
Index 12: walk-left-0	Index 14: walk-left-2	Index 16: walk-right-0
Index 13: walk-left-1	Index 15: walk-left-3	Index 17: walk-right-1
Index 18: walk-right-2		
Index 19: walk-right-3		

MAC-1-F3 | Draw the Minimum Viable Avatar Sprite Set [MAC]

These 15 PNG files are the minimum required for Phase 1 alpha. Each file is a 1280×96 pixel transparent PNG drawn in LibreSprite. Set the canvas size to **1280 × 96 px** in LibreSprite and draw frame by frame, advancing 64 pixels right for each new frame.

Drawing approach: Start with `body/body-skin-light.png` and draw the full character silhouette — this establishes the proportions every other layer must match. All clothing and hair layers must be drawn as transparent everywhere except the region they cover. Test alignment by opening multiple layers in LibreSprite and stacking them as separate Layers inside one document.

Category	File Path (relative to src/assets/sprites/)	Count
Body	body/body-skin-light.png, body/body-skin-medium.png, body/body-skin-dark.png	3
Eyes	eyes/eyes-default.png, eyes/eyes-round.png	2

Category	File Path (relative to src/assets/sprites/)	Count
Hair	hair/hair-short-01.png, hair/hair-short-02.png, hair/hair-long-01.png	3
Tops (shirts)	clothing/shirt-white.png, clothing/shirt-black.png, clothing/shirt-blue.png	3
Bottoms (pants)	clothing/pants-blue.png, clothing/pants-black.png	2
Feet (shoes)	clothing/shoes-white.png, clothing/shoes-black.png	2
Total	15	

Export each file from LibreSprite: **File** → **Export** → **PNG**. Confirm the exported size is 1280 × 96 and that the background is transparent (checkerboard pattern in LibreSprite = transparent).

MAC-1-F4 | Pack All Layers Into a Phaser Atlas with TexturePacker [MAC]

Open TexturePacker. Click **Add Sprites** and select all 15 PNG files. Configure the following settings before publishing:

Setting	Value	Why
Data Format	Phaser 3 (JSON Hash)	Required for Phaser's atlas loader
Texture Format	RGBA8888	Full colour + transparency
Max Texture Size	2048 × 2048	Safe limit for all devices including mobile
Algorithm	MaxRects — Best Short Side Fit	Efficient packing for long horizontal strips
Allow Rotation	DISABLED	Rotation breaks frame index order — never

Setting	Value	Why
		enable this
Trim	Enabled	Removes transparent border pixels; improves performance
Extrude	1	Prevents tile bleeding at seams when GPU interpolates

Set output file paths:

Texture: apps/client/src/assets/sprites/avatar/atlas.png Data: apps/client/src/assets/sprites/avatar/atlas.json

Click **Publish Sprite Sheet**. The resulting `atlas.json` maps every source filename (e.g., `hair/hair-short-01`) and frame index to its pixel coordinates within `atlas.png`. Phaser uses this JSON to reconstruct and play any frame by name during animation.

MAC-1-F5 | Register Avatar Animations in Phaser (AvatarAnimations.ts) [MAC]

Create `src/entities/AvatarAnimations.ts`. Animations are registered globally into the Phaser animation manager — they only need to be created once (the guard `if (scene.anims.exists('walk-down')) return;` prevents duplicate registration). All avatar sprites on screen share these global animation definitions.

```
export function createAvatarAnimations(scene: Phaser.Scene, layerKeys: string[]) {
  const directions = ['down', 'up', 'left', 'right'];
  const frameOffsets: Record<string, number> = {
    down: 4, up: 8, left: 12, right: 16
  };
  const idleOffsets: Record<string, number> = {
    down: 0, up: 1, left: 2, right: 3
  };
  if (scene.anims.exists('walk-down')) return; // Guard: create only once
  directions.forEach(dir => {
    // Walking animation: 4 frames at 8 fps, looping
    scene.anims.create({
      key: `walk-${dir}`,
      frames: [
        { key: 'avatars', frame: frameOffsets[dir] },
        { key: 'avatars', frame: frameOffsets[dir] + 1 },
        { key: 'avatars', frame: frameOffsets[dir] + 2 },
        { key: 'avatars', frame: frameOffsets[dir] + 3 }
      ],
      frameRate: 8,
      repeat: -1,
    });
    // Idle animation: single frame, no loop
    scene.anims.create({
      key: `idle-${dir}`,
      frames: [
        { key: 'avatars', frame: idleOffsets[dir] }
      ],
      frameRate: 1,
      repeat: 0,
    });
  });
}
```

IMPLEMENTATION NOTE

The frame references above use numeric indices for clarity. In your actual implementation, reference frame names directly from `atlas.json` (e.g., `body_0`) to ensure the correct frames are selected per layer. Match your `atlas.json` frame naming exactly — the exact naming depends on your `LibreSprite` file names and `TexturePacker`'s output settings.

MAC-1-F6 | Create the Avatar Entity Class (`src/entities/Avatar.ts`) [MAC]

The `Avatar` class extends `Phaser.GameObjects.Container`. A `Container` is a parent object that groups child game objects — all layer sprites and the name tag are children of the `Container`. When you move the `Container`, all children move with it. The render order of children within the `Container` matches the order they are added, establishing the correct visual stacking (body → eyes → clothing → hair → head items). Create `src/entities/Avatar.ts`

```
import Phaser from 'phaser'; import type { AvatarData, PlayerState } from '@havenworld/shared'; // Render layers from bottom to top const LAYER_ORDER = [ 'body', 'eyes', 'clothing_legs', 'clothing_body', 'clothing_feet', 'clothing_back', 'clothing_head', 'clothing_face', ]; export class Avatar extends Phaser.GameObjects.Container { public playerId: string; private layers = new Map<string, Phaser.GameObjects.Sprite>(); private nameTag: Phaser.GameObjects.Text; private speechBubble: Phaser.GameObjects.Text | null = null; private speechTimer: Phaser.Time.TimerEvent | null = null; private facing = 'down'; constructor(scene: Phaser.Scene, player: PlayerState) { super(scene, player.x, player.y); this.playerId = player.id; this.buildLayers(player.avatar); this.nameTag = scene.add.text(0, 52, player.username, { fontSize: '11px', color: '#ffffff', stroke: '#000', strokeThickness: 3, }).setOrigin(0.5, 0); this.add(this.nameTag); scene.add.existing(this); } private buildLayers(data: AvatarData) { const frameMap: Record<string, string | null> = { body: `body/body-${data.skinTone}`, eyes: data.eyeStyle ? `eyes/${data.eyeStyle}` : null, clothing_body: data.outfitBody ? `clothing/${data.outfitBody}` : null, clothing_legs: data.outfitLegs ? `clothing/${data.outfitLegs}` : null, clothing_feet: data.outfitFeet ? `clothing/${data.outfitFeet}` : null, clothing_back: data.outfitBack ? `clothing/${data.outfitBack}` : null, clothing_head: data.outfitHead ? `clothing/${data.outfitHead}` : null, clothing_face: data.outfitFace ? `clothing/${data.outfitFace}` : null, }; LAYER_ORDER.forEach(layer => { const frame = frameMap[layer]; if (!frame) return; const sprite = this.scene.add.sprite(0, 0,
```

```
'avatars', `${frame}-idle-down`);      this.layers.set(layer, sprite);
this.add(sprite);    });    }    playWalk(direction: string)
{      this.facing = direction;      this.layers.forEach(s => s.play(`walk-$
{direction}`, true));    }    stopWalk() {      this.layers.forEach(s =>
s.play(`idle-${this.facing}`, true));    }    showSpeechBubble(text: string) {
this.speechBubble?.destroy();      this.speechTimer?.remove();
this.speechBubble = this.scene.add.text(0, -80, text, {      fontSize:
'12px',      color:      '#fff',      backgroundColor: '#00000088',
padding:      { x: 8, y: 4 },      wordWrap:      { width:
160 },    }).setOrigin(0.5, 1);      this.add(this.speechBubble);
this.speechTimer = this.scene.time.delayedCall(4000, () =>
{      this.speechBubble?.destroy();      this.speechBubble = null;    });
}    updateAvatar(data: AvatarData) {      this.layers.forEach(s =>
s.destroy());      this.layers.clear();      this.remove(this.nameTag);
this.buildLayers(data);      this.add(this.nameTag);    } }
```

5.7 Room Map Creation with Tiled [MAC]

All HavenWorld room maps are created in the **Tiled Map Editor** — a free, open-source orthogonal and isometric map editor used widely in indie game development. Tiled exports maps as JSON files that Phaser 3 can load directly. Each map has four layers with defined purposes. The layer naming convention (**Ground**, **Walls**, **Objects**, **Overlay**) must be followed exactly, because **RoomScene.ts** references these layers by name.

MAC-1-G1 | Install and Set Up Tiled Map Editor [MAC]

Download Tiled from thorbjorn.itch.io/tiled (free / pay-what-you-want). Install and open Tiled. Before creating maps, configure: **Edit** → **Preferences** → **General** → **Tile Layer Format: CSV**. CSV format is required for Phaser's tilemap loader; the default Base64 format will cause loading errors.

Create the outdoor and indoor tileset images in LibreSprite before starting any maps:

Tileset File	Canvas Size	Tile Size	Grid	Key Tiles to Draw
tilesets/outdoor.png	256 × 256 px	32 × 32 px	8 × 8	Dark grass, light grass path, stone

Tileset File	Canvas Size	Tile Size	Grid	Key Tiles to Draw
				path, dirt, wall/hedge, water, flower bed, sand/gravel, plus decorative variants
tilesets/ indoor.png	256 × 256 px	32 × 32 px	8 × 8	Wood floor, carpet, stone floor, plain wall, wall with window, door, table top, chair seat, plus decorative variants

Layer conventions for all maps:

Layer Name	Purpose	Collision	Depth
Ground	Floor tiles — covers every tile of the map	None	Below avatars
Walls	Walls, obstacles, solid objects	collides: true (custom property on layer)	Below avatars
Objects	Non-blocking decorative items (signs, plants, benches)	None	Below avatars
Overlay	Items that render above the player (lamp tops, awning edges)	None	depth: 100 — set in Phaser

TIP — Setting the Collision Property in Tiled

To enable collision detection in Phaser, right-click the `Walls` layer in Tiled → **Layer Properties** → click the **+** icon → add a *bool* property named `collides`, set to `true`. Phaser's `setCollisionByProperty({ collides: true })` reads this property automatically.

MAC-1-G2 | Create the Lobby Map — 20 × 15 Tiles [MAC]

In Tiled: **File** → **New Map**. Settings: Orientation = Orthogonal, Tile size = 32 × 32 px, Map size = 20 tiles wide × 15 tiles tall (640 × 480 pixels total). Add the outdoor tileset: **Map** → **Add Tileset** → **From File** → **outdoor.png**, tile width 32, tile height 32. Design as an entry hall with stone floor, walls at map edges, welcome sign, benches along the sides, and clear path exits. Export: **File** → **Export As** → **apps/client/src/assets/maps/lobby.json**

MAC-1-G3 | Create Town Square Map — 30 × 20 Tiles [MAC]

New map: 30 × 20 tiles (960 × 640 pixels). Use outdoor tileset. Design as an open outdoor town center: grass background, stone paths in a cross pattern, a fountain or central feature, trees and bushes framing the edges, benches. Export as `town-square.json`

MAC-1-G4 | Create Haven Park Map — 25 × 18 Tiles [MAC]

New map: 25 × 18 tiles. Use outdoor tileset. Design as a relaxed park: mostly grass with winding path, clusters of trees (collision on trunk tiles; open canopy tiles in Overlay layer so they render above players), a small pond area. Export as `park.json`

MAC-1-G5 | Create The Cozy Café Map — 15 × 12 Tiles [MAC]

New map: 15 × 12 tiles. Use indoor tileset. Design as an intimate interior: wood floor, tables and chairs in a café layout, a service counter at one end, warm decor items in the Objects layer. Export as `cafe.json`

MAC-1-G6 | Create Personal Room Template Map — 10 × 8 Tiles [MAC]

New map: 10 × 8 tiles. Use indoor tileset. Design as a blank personal room: carpet or wood floor, plain walls, a door tile on one side. Keep it intentionally minimal — players decorate it themselves with furniture. This JSON is served as the starting template for all new players whose personal room has never been customized. Export as `personal-room.json`

MAC-1-G7 | Load Tilemaps in Phaser (RoomScene.ts) [MAC]

In `RoomScene.create()`, after receiving the `roomId` from scene launch data:

```
const map = this.make.tilemap({ key: `map-${roomId}` }); // Both tilesets
are added – only one will apply per room const outdoorTiles =
map.addTilesetImage('outdoor', 'tileset-outdoor'); const indoorTiles =
map.addTilesetImage('indoor', 'tileset-indoor'); const tileset =
outdoorTiles ?? indoorTiles!; // use whichever applies to this room const
groundLayer = map.createLayer('Ground', tileset, 0, 0); const wallsLayer
= map.createLayer('Walls', tileset, 0, 0); const objectsLayer =
map.createLayer('Objects', tileset, 0, 0); const overlayLayer =
map.createLayer('Overlay', tileset, 0, 0); // Enable collision on all tiles
in Walls layer with collides: true property
wallsLayer!.setCollisionByProperty({ collides: true }); // Camera: prevent
scrolling past the tilemap edges this.cameras.main.setBounds(0, 0,
map.widthInPixels, map.heightInPixels); // Overlay layer renders above all
avatars (depth 100 > avatar Y-based depth ~0-768)
overlayLayer!.setDepth(100);
```

5.8 Click-to-Move with EasyStar.js Pathfinding [MAC]

HavenWorld uses **EasyStar.js** for A* pathfinding on the tile grid. When a player clicks a destination tile, EasyStar finds the shortest walkable path from the avatar's current tile to the destination tile, avoiding any collision tiles. The path is returned as an array of tile coordinates, and the avatar's update loop walks tile-by-tile along that path. This approach is common in social world games because it is predictable, easy to synchronize over the network (the server only needs to know the current tile position and direction), and works well with tile-based maps.

MAC-1-H1 | Build the Walkability Grid from the Tilemap [MAC]

After creating all layers in `roomScene.create()`, read the Walls layer tile-by-tile to build the EasyStar grid. A value of `1` means blocked; `0` means walkable.

```
import EasyStar from 'easystarjs'; const easyStar = new EasyStar.js(); const
grid: number[][] = []; for (let row = 0; row < map.height; row++) { const
rowArr: number[] = []; for (let col = 0; col < map.width; col++)
{ const tile = wallsLayer!.getTileAt(col, row); // 1 = blocked
(collision tile), 0 = walkable (no collision tile) rowArr.push(tile &&
tile.properties?.collides ? 1 : 0); } grid.push(rowArr); }
easyStar.setGrid(grid); easyStar.setAcceptableTiles([0]); // 0 = walkable
easyStar.enableDiagonals(); // allow diagonal movement between tiles
easyStar.disableCornerCutting(); // prevent cutting through wall corners
diagonally
```

MAC-1-H2 | Handle Click-to-Move Input [MAC]

In `roomScene.create()`, declare the path state variables and set up the pointer input handler. The key conversion step is going from screen coordinates (where the mouse was clicked relative to the viewport) to world coordinates (accounting for camera scroll offset), then to tile coordinates.

```
let walkPath: { x: number; y: number }[] = []; let pathIndex = 0;
this.input.on('pointerdown', (pointer: Phaser.Input.Pointer) => { //
Convert screen → world coordinates (accounts for camera offset + zoom)
const worldPoint = this.cameras.main.getWorldPoint(pointer.x,
pointer.y); // Convert world pixel coordinates → tile coordinates const
destTileX = Math.floor(worldPoint.x / 32); const destTileY =
```

```

Math.floor(worldPoint.y / 32); // Current tile position of the local
avatar  const startTileX = Math.floor(localAvatar.x / 32);  const
startTileY = Math.floor(localAvatar.y / 32); // Queue a pathfinding
request (asynchronous – result arrives via callback)
easyStar.findPath(startTileX, startTileY, destTileX, destTileY, (path) => {
if (path && path.length > 1) {      walkPath = path.slice(1); // skip
index 0 (the starting tile – already there)      pathIndex = 0;      }  });
// Must call calculate() to actually process the queued request
easyStar.calculate(); });

```

MAC-1-H3 | Walk the Avatar Along the Path in update() [MAC]

Each frame in `RoomScene.update()`, advance the avatar toward the next tile in the path. When the avatar arrives within 2 pixels of the target tile center, snap it to exact center and advance to the next tile. When the full path is exhausted, stop the walk animation and clear the path array. The `player:move` socket event is emitted on each tile step so the server can relay the updated position to all other players in the room.

```

// In RoomScene.update():  if (walkPath.length > 0 && pathIndex <
walkPath.length) {  const target = walkPath[pathIndex];  const targetX =
target.x * 32 + 16; // center of tile (32px wide, 16px offset)  const
targetY = target.y * 32 + 16;  const dx = targetX - localAvatar.x;
const dy = targetY - localAvatar.y;  const dist = Math.sqrt(dx * dx + dy *
dy);  if (dist < 2) {  // Arrived at this tile: snap to exact center
localAvatar.setPosition(targetX, targetY);  pathIndex++;  if
(pathIndex < walkPath.length) {  // Determine direction to the next tile
const next = walkPath[pathIndex];  const direction =  next.x >
target.x ? 'right' :  next.x < target.x ? 'left' :  next.y >
target.y ? 'down' : 'up';  localAvatar.playWalk(direction);  //
Broadcast this step to the server
socketService.emit(SOCKET_EVENTS.PLAYER_MOVE, {  x:
localAvatar.x,  y:  localAvatar.y,  direction,
roomId:  currentRoomId,  });  } else {  // Reached final
destination  localAvatar.stopWalk();  walkPath = [];  }  }
else {  // Still moving toward target tile at fixed pixel speed  const
speed = 2;  localAvatar.x += (dx / dist) * speed;  localAvatar.y += (dy
/ dist) * speed;  } } // EasyStar must be called each frame to process any
queued pathfinding requests easyStar.calculate();

```

5.9 Avatar Customizer UI [MAC]

The avatar customizer is a pure HTML/TypeScript module — not a Phaser object. It reads the player's current inventory from the REST API and renders a tabbed grid of wearable items. When a player clicks an item to try it on, the selection is

immediately broadcast to the server via the `avatar:update` socket event so other players in the room see the preview in real time. The player confirms the change by clicking Save, which persists the selection to the database via the `PUT /api/users/me/avatar` endpoint.

MAC-1-I1 | Create AvatarCustomizer Module (src/ui/AvatarCustomizer.ts) [MAC]

```
import { authService } from '../services/auth'; import { socketService }
from '../services/socket'; import { SOCKET_EVENTS } from
'@havenworld/shared'; export class AvatarCustomizer { private inventory:
any[] = []; private current: Record<string, string | null> =
{}; async open(currentAvatar: any) { this.current = { ...currentAvatar
}; const res = await fetch('/api/users/me/inventory', { headers:
{ Authorization: `Bearer ${authService.getToken()}` }, });
this.inventory = await res.json(); this.renderTabs();
document.getElementById('avatar-panel')!.style.display = 'flex'; }
close() { document.getElementById('avatar-panel')!.style.display = 'none'; }
private renderTabs() { const tabs = [ { label: 'Hair', cat:
'HAIR' }, { label: 'Eyes', cat: 'EYES' },
{ label: 'Tops', cat: 'SHIRT' }, { label: 'Bottoms',
cat: 'PANTS' }, { label: 'Shoes', cat: 'SHOES' }, {
label: 'Hats', cat: 'HAT' }, { label: 'Accessories', cat:
'ACCESSORY' }, ]; const tabEl = document.getElementById('avatar-
tabs')!; tabEl.innerHTML = tabs.map(t => `<button class="avatar-
tab" data-cat="${t.cat}">${t.label}</button>`).join('');
tabEl.querySelectorAll('.avatar-tab').forEach(btn =>
{ btn.addEventListener('click', () =>
this.renderGrid(btn.getAttribute('data-cat')!); });
this.renderGrid('HAIR'); // default to Hair tab on open } private
renderGrid(category: string) { const items = this.inventory.filter(i =>
i.category === category); const grid = document.getElementById('avatar-
items-grid')!; grid.innerHTML = items.length ? items.map(item => `
<div class="avatar-item ${this.isEquipped(item, category) ? 'equipped' : ''}"
data-item-id="${item.itemId}" data-cat="${category}"> <div
class="avatar-item-sprite" data-sprite="${item.spriteKey}"></div>
<span class="avatar-item-name">${item.name}</span> </div>
`).join('') : '<p style="color:#aaa; padding:16px;">No items in this
category yet.</p>'; grid.querySelectorAll('.avatar-item').forEach(el =>
{ el.addEventListener('click', () =>
{ this.tryOn(
el.getAttribute('data-item-id')!,
el.getAttribute('data-cat')!); }); }); } private
isEquipped(item: any, category: string): boolean { const catToKey:
Record<string, string> = { HAIR: 'hairStyle', SHIRT: 'outfitBody',
PANTS: 'outfitLegs', SHOES: 'outfitFeet', HAT: 'outfitHead', ACCESSORY:
'outfitFace', }; return this.current[catToKey[category]] ===
item.itemId; } private tryOn(itemId: string, category: string)
{ const catToKey: Record<string, string> = { HAIR: 'hairStyle',
SHIRT: 'outfitBody', PANTS: 'outfitLegs', SHOES: 'outfitFeet', HAT:
'outfitHead', ACCESSORY: 'outfitFace', };
this.current[catToKey[category]] = itemId; // Broadcast live preview to
all players in the room socketService.emit(SOCKET_EVENTS.AVATAR_UPDATE,
```

```
this.current);    this.renderGrid(category); // re-render grid to show
'equipped' state } async save() {    await
fetch('/api/users/me/avatar', {    method:    'PUT',    credentials:
'include',    headers: {    'Content-Type': 'application/json',
Authorization: `Bearer ${AuthService.getToken()}`,    },    body:
JSON.stringify(this.current),    });    this.close();    } }
```

MAC-1-I2 | Wire Up the Avatar Panel Buttons [MAC]

In `src/main.ts`, after creating the Phaser game instance, import and initialize the customizer and wire all three buttons:

```
import { AvatarCustomizer } from './ui/AvatarCustomizer'; const
avatarCustomizer = new AvatarCustomizer(); document.getElementById('btn-
avatar')?.addEventListener('click', () => {    const currentAvatar =
AuthService.getUser()?.avatar ?? {};
avatarCustomizer.open(currentAvatar); }); document.getElementById('avatar-
save')?.addEventListener('click', () =>    avatarCustomizer.save() );
document.getElementById('avatar-close')?.addEventListener('click', () =>
avatarCustomizer.close() );
```

5.10 Phase 1 Mac Completion Checklist [MAC]

Work through this checklist top-to-bottom before proceeding to Part 4 (Linux/server work). Every item must be checked before the Phase 1 Mac milestone is considered complete. Items that depend on the server being live (e.g., socket connection tests) can be partially verified with stub responses and fully verified after Part 4 is complete.

Project Bootstrap

- [] Vite + Phaser 3 scaffolded in `apps/client` using `ant-class` template
- [] Client dependencies installed: `phaser`, `socket.io-client`, `jquery`, `asvstar`
- [] `vite.config.ts` configured with proxy, manual chunk splitting, and static copy plugin
- [] `apps/server/package.json` and `server.config.json` created on Mac
- [] All server dependencies installed (`express`, `socket.io`, `phaser3/Client`, `crypto`, `jsonwebtoken`, etc.)
- [] `env.development` and `env.production` created in `apps/client`
- [] Both `.env` files added to `.gitignore` — not committed to Git
- [] `VITE_SERVER_URL` added to Cloudflare Pages environment variables (Production)

Shared Package

- [] `packages/shared` initialized with correct `package.json` (name, exports)
- [] All 14 `SOCKET_EVENT` constants defined in `src/events.ts` with `as const`
- [] All 5 TypeScript interfaces defined in `src/types.ts`: `GameData`, `PlayerStats`, `GameData`, `ChatMessage`, `FurnitureStats`
- [] `src/index.ts` re-exports both `events` and `types`
- [] `tsconfig.json` added to client and server `tsconfig`
- [] `npm install` run from repo root — workspace symlink created and resolvable

Phaser Scenes

- [] `main.js` created — Phaser `GameConfig` defined, all 5 scenes registered
- [] `BootScene`: loads all atlases (`textures`, `fontTextures`), tilemaps (5 maps), and tilesets (outdoor, indoor) in `preload()`
- [] `UIScene`: reveals `roomListPanel` correctly on `start()`
- [] `LobbyScene`: fetches room list from API, renders room cards, click handler starts `RoomScene` and launches `UIScene`
- [] `RoomScene`: all 11 socket event listeners stubbed and connected
- [] `RoomScene`: tilemap loads with correct 4 layers, camera bounds and follow set
- [] `UIScene`: chat send (button and Enter key) and `CHAT_MESSAGE` receive both work
- [] `UIScene`: rate-limit error message displays and auto-clears after 2 seconds

Auth and Socket Services

- [] `src/services/auth.js` created — `register`, `login`, `logout`, `refresh`, `getToken` all implemented
- [] Access token stored in module variable (not `localStorage`)
- [] Token auto-refresh `setInterval` fires every 13 minutes
- [] `src/services/socket.js` created — singleton `socketService` class
- [] `connect` guards against duplicate connections (`socket.isConnected` check)
- [] `emit`, `on`, `off`, `disconnect`, `isConnected` all implemented and exported

HTML Shell and CSS

- [] `index.html` created with all 7 panels: `game-container`, `login-pane`, `lobby-pane`, `chat-pane`, `avatar-pane`, `room-pane`, `player-card`
- [] All panels have `display: none` as default (except `game-container`)
- [] Login form tab switching (Login / Register tabs) works correctly
- [] Register form includes invite code field
- [] `chat-input` has `maxLength=200` attribute
- [] `index.html` created — dark navy (`#0d0d1a`) body, teal (`#4ecdc4`) accents, all panels styled
- [] Input focus states use teal border (`border: 1px solid #4ecdc4`)
- [] Room cards have hover lift effect (`transform: translateY(-40%)`)

Avatar Art Pipeline

- [] Libresprite installed on Mac from <https://libresprite.com/Libresprite/Libresprite/release>
- [] TexturePacker (free tier) installed on Mac
- [] Frame specification established and documented: 64×96 px, 20 frames per layer, horizontal strip order confirmed
- [] All 15 layer PNGs drawn: 3 body skins, 2 eyes, 3 hair, 3 shirts, 2 pants, 2 shoes
- [] All PNGs exported at 1280×96 px with transparent background
- [] TexturePacker: Allow Rotation **DISABLED**, Data Format = Phaser 3 (JSON Hash)
- [] `atlas.png` and `atlas.json` published to `src/assets/sprites/avatar`
- [] Phaser atlas loads in BootScene without console errors
- [] `AvatarAnimations` created — `walk-down/up/left/right` and `idle-down/up/left/right` registered
- [] `Avatar` Container class complete: `Avatar`, name tag, speech bubble, `playwalk`, `stopwalk`, `updateAvatar`

Tiled Maps

- [] Tiled installed on Mac
- [] Tiled Preferences: Tile Layer Format = CSV
- [] `outdoor.png` tileset drawn in LibreSprite (256×256 px, 32×32 tiles, 8 key tile types)
- [] `indoor.png` tileset drawn in LibreSprite (256×256 px, 32×32 tiles, 8 key tile types)
- [] `lobby.iso` (20×15 tiles) created and exported to `src/assets/maps/`
- [] `down-square.iso` (30×20 tiles) created and exported
- [] `back.iso` (25×18 tiles) created and exported
- [] `gate.iso` (15×12 tiles) created and exported
- [] `personals-room.iso` (10×8 tiles) created and exported
- [] All maps have 4 layers: `background`, `background2`, `background3`, `background4` (exact names)
- [] `background` layer in each map has custom bool property `noTides = true` set in Tiled
- [] All 5 maps load in Phaser with no console errors (missing tileset, wrong layer name, etc.)
- [] Overlay layer renders above avatar sprites (`setDepth(100)` confirmed visually)

Click-to-Move Pathfinding

- [] EasyStar walkability grid built correctly from Walls layer (0 = walkable, 1 = blocked)
- [] Diagonals enabled; corner cutting disabled
- [] Pointer click handler converts screen → world → tile coordinates correctly
- [] `easyStar.findPath()` called, followed immediately by `easyStar.calculate()`
- [] Avatar walks tile-by-tile along returned path in `update()` loop
- [] Avatar snaps to tile center (`tile * 32 + 16`) on arrival at each tile
- [] Walk animation plays in correct direction per tile step
- [] Idle animation plays when destination reached
- [] `PLAYER_MOVE` socket event emitted on each tile step with current x, y, direction, roomId
- [] `easyStar.calculate()` called every frame in `update()`
- [] Clicking a wall tile produces no movement (EasyStar returns null path)

Avatar Customizer

- [] `AvatarCustomizer.vue` created with `open()`, `close()`, `renderAus()`, `renderGInt()`, `tryOn()`, `save()`
- [] Fetches inventory from `GET /api/users/me/inventory` with auth header
- [] All 7 category tabs render and switch grid content correctly
- [] Currently equipped item shown with `equipped` CSS class highlighting
- [] Try-on emits `AVATAR_UPDATE` socket event immediately (live preview for other players)
- [] Save button calls `PUT /api/users/me/avatar` and closes panel
- [] `AvatarAvatar`, `AvatarSave`, `AvatarClose` all wired in `main`

Continuing in Part 4 of 4 — Phase 1 Linux: Server, Auth, Socket Engine & Deploy

Part 4 covers all Phase 1 Linux/Oracle VM tasks: the full Prisma schema and database migrations, the Node.js + Express game server entry point, the complete authentication system (register with invite code, email verification via Resend, login, access token refresh, and logout), the Socket.io real-time engine with `RoomManager` and `ModerationService`, all REST API routes (`auth`, `room`, `user`), PM2 process management and production deployment, the invite code generation script for alpha access control, and the final **Alpha-Ready Gate Checklist** that confirms all systems are green before inviting the first testers.